arXiv:2506.12282v1 [cs.DC] 14 Jun 2025

Towards Energy-Efficient Distributed Agreement

Hugo Mirault
Augusta University, Augusta, Georgia, USA

Peter Robinson
Augusta University, Augusta, Georgia, USA

Abstract

We study fault-tolerant consensus in a variant of the synchronous message passing model, where, in each round, every node can choose to be awake or asleep. This is known as the sleeping model (Chatterjee, Gmyr, Pandurangan PODC 2020) and defines the awake complexity (also called *energy complexity*), which measures the maximum number of rounds that any node is awake throughout the execution. Only awake nodes can send and receive messages in a given round and all messages sent to sleeping nodes are lost. We present new deterministic consensus algorithms that tolerate up to $f < n$ crash failures, where n is the number of nodes. Our algorithms match the optimal time complexity lower bound of $f + 1$ rounds. For multi-value consensus, where the input values are chosen from some possibly large set, we achieve an energy complexity of $\mathcal{O}(\lceil f^2/n \rceil)$ rounds, whereas for binary consensus, we show that $\mathcal{O}(\lceil f/\sqrt{n} \rceil)$ rounds are possible.

2012 ACM Subject Classification Theory of computation → Distributed algorithms

Keywords and phrases Distributed computing, Crash faults, Consensus, Energy complexity, Sleeping model

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Reaching agreement in a network of distributed processes in the presence of faults is a fundamental problem in distributed computing with numerous applications, including blockchain networks and synchronization problems; e.g., see [Xu et al.(2023), Attiya and Welch(2004), Lynch(1996)]. In the consensus problem [Pease et al.(1980), Lamport et al.(1982)], there is a set of players (or nodes) that communicate with each other by sending point-to-point messages. Each player holds an initial value, and the goal is that, eventually, all of them decide on the same value.

While the problem can be solved in just a single round of communication assuming that all players operate correctly, achieving consensus becomes significantly more challenging when taking into account that some of the players may unexpectedly fail throughout the execution. The seminal result of [Fischer et al.(1985)] showed that consensus is impossible with a deterministic algorithm in asynchronous networks, when even just a single player may crash. Moreover, [Dolev and Strong(1983)] proved that $f + 1$ rounds are indeed necessary in the synchronous setting, when considering up to f crash failures, and a simple algorithm matches this lower bound on the time complexity.

Even though the time complexity of deterministic consensus has been resolved, the actual number of rounds that a player needs to actively participate in the protocol is less clear. This motivates studying consensus in the sleeping model, where, in every round, each player chooses to either be asleep or awake, and the *energy complexity* of the algorithm is defined as the maximum number of awake rounds over all players in the worst case. The sleeping model was first introduced in [Chatterjee et al.(2020)] for studying the (average) energy complexity[1] of computing a maximal independent set and has since received

[1] The terms "energy complexity" and "awake complexity" are used interchangeably in the literature. In

© Hugo Mirault and Peter Robinson;
licensed under Creative Commons License CC-BY 4.0

significant attention recently; e.g., see [Dufoulon et al.(2023), Ghaffari and Trygub(2024), Ghaffari and Portmann(2023), Hourani et al.(2022), Barenboim and Maimon(2021)]. The sleeping model can be interpreted as a simplified variant of the class of models studied in [Chang et al.(2018), Chang et al.(2020)], where the main focus is on wireless networks.

1.1 Our Contributions

We establish the first energy efficient algorithms for consensus under crash faults in the sleeping model, while still obtaining optimal time complexity. In more detail, we present the following new deterministic algorithms:

- Multi-value consensus can be solved with an energy complexity of $\mathcal{O}(\frac{f^2}{n})$ rounds, a time complexity of $f+1$, while sending at most $\mathcal{O}(f^3+nf)$ messages.
- Binary consensus can be solved with an energy complexity of $\mathcal{O}(\frac{f}{\sqrt{n}})$ rounds, a time complexity of $f+1$, and with a message complexity of $\mathcal{O}(n^2)$.

We point out that none of the prior works in the sleeping model considers faulty nodes, and thus, our results also shed some light on the impact of faults on the energy complexity.

2 Model and Preliminaries

We consider the standard synchronous message-passing model of distributed computing, where n players $p_0, \ldots, p_{n-1}$ are connected in a clique network. Each player can choose to either *sleep* or be *awake* in each round, which is known as the *sleeping model*. If a player p is asleep in some round, then any message sent to p in this round is lost; in particular, p does *not* receive these messages upon its next wake-up. In this work, we extend the above model with *crash faults*, which means that an adversary can crash up to f players throughout the execution, for some parameter $f < n$. As we only consider deterministic algorithms, we assume that the adversary is omniscient. If a player p *crashes* in round r, then only an arbitrary subset of messages sent by p are guaranteed to be delivered. Moreover, p does not perform any computation in any future round after crashing. Any player who does not crash in rounds $1, \ldots, r-1$ is said to be *alive in round* r If a player never crashes, we say that it is *non-faulty*.

2.1 Complexity measures

We quantify the performance of our algorithms in terms of the *message complexity*, i.e., the total number of messages sent in the worst case, and the *time complexity*, which captures the worst-case number of rounds until all players have terminated. In the sleeping model, we consider two measures of energy consumption:

- the *(worst case) energy complexity* is the maximum number of rounds (taken over all players) that a player is awake in the worst case execution;

2.2 Consensus

Every player starts with an input value that is assigned from a set D of integers. In Section 3, we assume that D is of size at most polynomial in n, whereas in Section 4, we restrict D to

this work, we use energy complexity.

be $\{0,1\}$. An algorithm solves consensus if, in any execution and for any input assignment, the following properties hold:

- *Termination*: Every non-faulty player eventually decides on some value.
- *Agreement*: If some player decides on v in an execution α, then every other player who decides must also decide on v.
- *Validity*: Any decision value must be the input value of some player (including the faulty ones).

As is standard in literature, for *binary consensus*, we assume that $\Delta = \{0,1\}$ is of size 2, while, *multi-value consensus* refers to the case where Δ is of larger size.

3 Multi-value Consensus

In the context of the sleeping model, it is pertinent to explore whether we can reduce the energy complexity at the expense of round complexity and/or message complexity. In this particular setting, we consider the case where the input value of each player can have value in any set of size ≥ 2. We now describe the high-level ideas of our algorithm and present the detailed pseudo-code in Algorithm 2. At the beginning of round 1, we fix f static committees $C_1, \ldots, C_f$, each containing $f+1$ members. The players use their IDs to determine which committee to join. That is, each player locally executes a function JoinCommittees(x,y), which outputs x committees of size y such that each player's ID is in at most $\lceil \frac{f(f+1)}{n} \rceil$ committees. Then, in Phase 1, which consists of round 1, all players awake

Algorithm 1 JoinCommittees(a,b)

Parameters : a : number of committees
b : size of committees

foreach $1 \leq i \leq a \cdot b$ **do**
  $c \leftarrow i \mod n$
  $d \leftarrow \lceil \frac{i}{b} \rceil$
  p_c join committee C_d

simultaneously and send their initial values to committee C_1. Each player $p \in C_1$ keeps track of the maximum received value in a variable Y that is initialized to p's input. Phase 2 consists of $f-1$ rounds, where all members of committee C_i are awake in rounds i and $i+1$. In round i, they receive the values sent by committee $i-1$ and each member computes its new maximum value locally. In round $i+1$, each player in C_i forwards this value to the next committee C_{i+1}. Finally, Phase 3 consists of round $f+1$, where everyone wakes up, and the committee members of C_f broadcast their values, and everyone decides on the maximum value that they have seen. See Figure 1 for a high-level overview of this process.

▶ **Theorem 1.** *Suppose that each player starts with an input value from a set of integers D. Algorithm 2 solves consensus in $f+1$ rounds, sends $\mathcal{O}(f^3+nf)$ messages of size $\mathcal{O}(\log|D|)$ bits, and achieves an energy complexity of $\mathcal{O}(\lceil f^2/n \rceil)$ rounds, if at most f players crash during the execution.*

3.1 Proof of Theorem 1

We first show that the algorithm correctly solves consensus. Every correct player decides on its variable Y after $f+1$ rounds, which ensures the claimed bound on the time complexity. By the code of the algorithm, Y is only updated to the input values of other players, and thus we obtain the following.

Algorithm 2 Multi Value Consensus. Code for player p.

```
   // Phase 1 (round 1)
 1 Player p wakes up in round 1:
 2 Locally compute JoinCommittees(f, f + 1)
 3 Y ← X
 4 Send Y to every player in C_1
 5 upon receiving values Y' do:
 6       Y ← max(Y ∪ (⋃ Y'))
   // Phase 2 (rounds 2, ..., f)
 7 For each round h ∈ [2, f] do:
 8 if p ∈ C_{h−1} then
 9 |   Wake up in round h
10 |   Send Y to every player in C_h
11 if p ∈ C_h then
12 |   Wake up in round h
13 |   upon receiving values Y' do:
14 |       Y ← max(Y ∪ (⋃ Y'))
   // Phase 3 (round f + 1)
15 Player p wakes up in round f + 1
16 if p ∈ C_f then
17 |   Broadcast Y to all
18 upon receiving values Y' do:
19       Y ← max(Y ∪ (⋃ Y'))
20 Decide on Y at the end of round f + 1
```

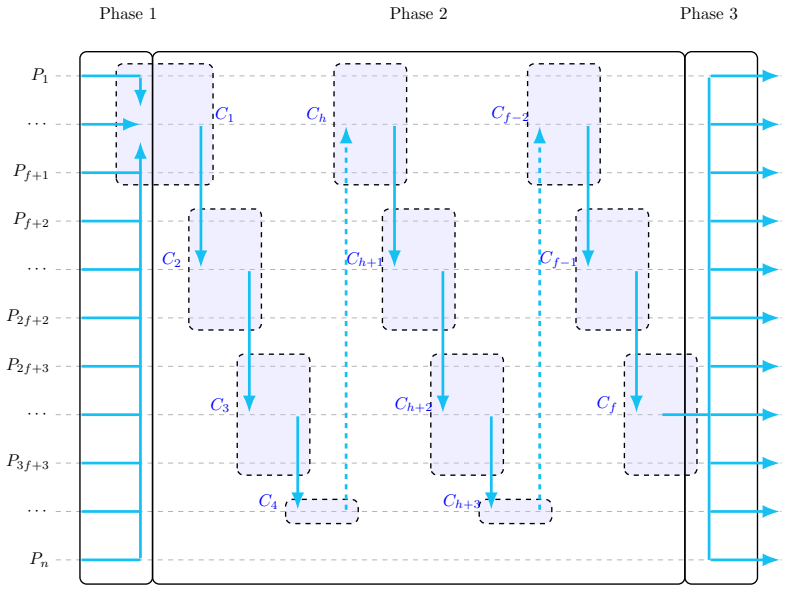


Figure 1 High-level Overview of Algorithm 2

▶ **Lemma 2.** *Algorithm 2 satisfies validity and terminates in $f + 1$ rounds.*

To show that the algorithm satisfies agreement, we need to prove several technical lemmas. Consider the set D of values decided by the players after round $f+1$ in some given execution, and let $\delta = \max(D)$. We say that *a player p has value v* if p's variable $Y = v$ at that point.

▶ **Lemma 3.** *Suppose that there exists some round $i \in [1, f + 1]$ with the property that there is an awake player p, who is alive in round i and sends a message with value $v_p > \delta$. Then no player decides on δ.*

Proof. Consider any round i that satisfies the premise of the claim. It follows that p successfully sends its message to all destinations. In the case that $i = f + 1$, this means that $p \in C_f$, and every player receives p's value in round $f + 1$, which is strictly greater than δ. Hence, none of the players will decide on δ. On the other hand, if $i \leq f$, then every alive member $p' \in C_i$ receives the message from p and updates its variable Y such that $Y > \delta$. Since not all members of C_i can crash, it follows that there is some alive player in C_i, who

is awake and alive in $i+1$ and who sends a value strictly greater than δ. Extending this argument inductively for all rounds $i, \ldots, f+1$, completes the proof of the lemma. ◀

▶ **Lemma 4.** *Consider any round $i \in [1, f]$ and let $m \geq \delta$ be the maximum value held by any player in C_i at the start of round $i+1$. Then, every alive player has a value of at most m at the beginning of round $i+1$.*

Proof. The statement trivially holds for any alive player in C_i. Now, consider some alive player $p \notin C_i$ and assume, towards a contradiction, that p's value is $v > m$ when it starts round $i+1$. First, consider the case where v was p's input value. This means that p sent v to every player in C_1 initially. Since p did not crash, every $p' \in C_1$ must have set Y to some value $v_{p'} \geq v$ at the end or round 1 and subsequently sends $v_{p'}$ to everyone in C_2 in round 2. Recall that every committee has at least one member who does not crash. Thus, it follows by induction that every player $q \in C_j$ $(j \geq 2)$ will receive a message with a value no smaller than v in round j from some alive player in C_{j-1} and update its own value accordingly. Consequently, q has set Y to a value strictly greater than m at the start of round $j+1$. In particular, this statement holds for $j = i$, which contradicts the assumption that m was the maximum value among players in C_i. Now suppose that v was not p's input value, which means that p must have been a member of some committee C_j and adopted v as its value in some round $j \leq i$. Since p did not crash in round $j+1$ and recalling that $v > \delta$, applying Lemma 3 yields a contradiction to the assumption that some player decided on δ. ◀

▶ **Lemma 5.** *Algorithm 2 satisfies agreement.*

Proof. Consider the earliest round $t \in [1, f+1]$ in which no player crashes. First, assume that $t = 1$. Then, it follows from Lemma 3 that no player has an input greater than δ. Thus, it follows from validity (see Lem. 2) that δ was the maximum input value, and hence every player in C_1 adopts δ in round 1. It follows that all alive members of every committee will have value δ, and thus every player decides on δ eventually. On the other hand, if $1 < t < f+1$, then Lemma 3 and the fact that someone decides on δ tell us that every alive player in C_{t-1} must have sent a value of at most δ in round t. Moreover, Lemma 4 ensures that all alive players (whether they are sleeping or awake) have a value of at most δ at the start of round $t+1$; let V denote this set of values. If $\max V < \delta$, then Lemma 4 implies us that every alive player has a value strictly less than δ. Since values are updated using the maximum rule, this contradicts the assumption that some player decided on δ, and hence it follows that some alive player $p \in C_t$ has the value δ, which is the maximum among all committee members. And, applying Lemma 4 guarantees that no greater values are held by players not in C_t. Since p does not crash, a simple inductive argument shows that δ is the only possible decision value. ◀

It remains to prove the claimed bound on the energy and message complexity: The first and the last phases require every player to be awake for a constant number of rounds. The second phase requires the players in each committee to be awake only in the two rounds during which their respective committee needs to receive or send messages. Since each player belongs to at most $\lceil \frac{(f+1)\cdot f}{n} \rceil$ committees, the energy complexity is $\mathcal{O}(\lceil f^2/n \rceil)$. During Phases 1 and 3, the algorithm sends $\mathcal{O}(f \cdot n)$ messages. In each round i of Phase 2, all players in committee C_{i-1} send a message to every player in committee C_i, which amounts to $\mathcal{O}(f^2)$ messages for each of the $f-1$ rounds, and thus the total number of messages is $\mathcal{O}(f^3 + f \cdot n)$.

4 Binary Consensus

In this section, we show how to improve over the energy complexity bounds obtained in Section 3, by restricting the domain of the input values to a single bit, while keeping the optimal time complexity of $f+1$ rounds. Recall that, in Algorithm 2, the energy complexity was dominated by the number of committees that each player is a member of, which in turn depends on the size of the individual committees, i.e., $f+1$. Here, we explore the natural strategy of reducing the committee size to achieve a lower energy complexity. While conceptually simple, having smaller committees entails that we can no longer rely on each committee having at least one alive member, requiring a player to spread its value among multiple committees instead. We compensate for this lack of information spreading across committees by making players aware of decisive information, some relay who keep spreading the decisive information to further committees. Assume players may choose 1 over 0 when they have the choice (i.e., they know at least 1 player holds each of those values as their initial value). This situation allows a player to know that everyone may decide 1 when he receives the value, and/or he will crash before the end of the procedure.

4.1 Description of the Algorithm

We now give a high-level overview of our algorithm; see Algorithm 3 for the detailed pseudo code: During initialization, we use procedure JoinCommittees (see Algorithm 1) to fix f different static committees $C_1, \ldots, C_f$, each of size $\lceil \sqrt{n} \rceil$ in such a way that, if a player p belongs to a committee C_i, then he does not belong to committees $C_{i+1}, \ldots, C_{i+\lceil \sqrt{n} \rceil+1}$; here we define $C_j = \emptyset$, for all $j > f$, to simplify the notation. A crucial property of being a member of committee C_i is that every $p \in C_i$ will wake up (at least) in round i, and thus we say that round i is the *awake round of committee* C_i. Each player has a local variable Y that is initialized with the input bit, in which we store the player's current decision estimate. Since our algorithm restricts itself to sending messages with bit value 1, called 1-*valued message*, we only ever update Y by setting $Y \leftarrow 1$ if it was 0 before receiving such a message.

Conceptually, the algorithm consists of three phases: Phase 1 consists only of round 1, in which every player p wakes up and, if p has input value $X = 1$, then they send a message with a single bit 1 to all members of committee C_1. Any member $q \in C_1$ who has $Y = 0$ and receives such a message becomes *active*, in the sense that q remains awake in the subsequent Phase 2 for the next $T = \left\lceil \frac{f+1}{\sqrt{n}} \right\rceil$ rounds.

In Phase 2, we propagate these 1-value messages across all other committees as follows. s $C_1, \ldots, C_{\lceil \frac{f+1}{\sqrt{n}} \rceil}$ in their respective awake rounds, i.e., in rounds $1, \ldots, \lceil \frac{f+1}{\sqrt{n}} \rceil$. If a player $p' \in C_i$ receives the value 1 for the first time upon waking up in round i (i.e., p' never received 1 before nor had 1 as their initial value), then p' will become *active*, and remain awake for the next $\lceil \frac{f+1}{\sqrt{n}} \rceil$ rounds, during which p' sends bit 1 to the next corresponding committees whose members are awake in the respective round.

When reaching round $f+1$ (i.e., Phase 3), all players wake up, and those having $Y = 1$ broadcast bit 1. Finally, every player decides on its current value of Y.

▶ **Theorem 6.** *Suppose that each player starts with a one-bit input value and that at most $f < n$ players crash. There is a deterministic algorithm that solves consensus in $f+1$ rounds, sends $\mathcal{O}(n^2)$ messages of constant size. Moreover, the algorithm achieves $\mathcal{O}(\lceil f/\sqrt{n} \rceil)$ energy complexity.*

Algorithm 3 Binary Value Consensus. Code for player p.

```
   // Phase 1 (round 1)
 1 Player p wakes up in round 1:
 2 Locally compute JoinCommittees(f, √n)
 3 Y ← 0; T ← 0
 4 if X = 1 then
 5 |   T ← ⌈(f+1)/√n⌉
 6 |   Send 1 to every player in committee C_1
 7 if p received value 1 and Y = 0 then
 8 |   Y ← 1; T ← ⌈(f+1)/√n⌉

   // Phase 2 (round 2, ..., f)
 9 For each round h ∈ [2, f] do:
10 if T > 0 then
11 |   Player p wakes up in round h
12 |   T ← T − 1
13 |   Send 1 to every player in C_h
14 if p ∈ C_h then
15 |   Player p wakes up in round h
16 |   if p received value 1 and Y = 0 then
17 |   |   Y ← 1; T ← ⌈(f+1)/√n⌉

   // Phase 3 (round f+1)
18 Player p always wakes up in round f+1
19 if Y = 1 or X = 1 then
20 |   Send 1 to all
21 if received value 1 or X = 1 then
22 |   Y ← 1
23 Decide on Y at the end of round f + 1
```

4.2 Proof of Theorem 6

▶ **Lemma 7.** *Algorithm 2 satisfies validity and terminates in $f + 1$ rounds.*

Proof. Every player decides on Y in round $f + 1$, hence termination is guaranteed in $f + 1$ rounds.

Next, we argue validity: In the case where every player starts with 0, no message is sent throughout the entire execution, causing all players to decide on their input value. Now consider the case where everyone starts with 1. Every player sets their decision value to their input value 1 in Line 19, and validity follows. ◀

We now focus on the agreement property: The cases where every player starts with the same input value are already covered by Lemma 7, and henceforth we can assume that there are players starting with 0 as well as with 1.

In our analysis, we keep track of a certain subset of the players that have adopted 1 as their value. To this end, we define a set P_r as follows: For round $r = 1$, we have $P_r = \{p_1\}$, where p_1 is the player with the largest ID who had an input value of 1. For $r > 1$, we define $P_r = P_{r-1} \cup R_{r-1}$, where R_r is the set of players that have received a message with value 1 in round $r - 1$. Note that P_r may not include every player who had 1 as their initial input value (unless it received a message with value 1), and some of the players in P_r may have already crashed by round r. In Phase 3 of Algorithm 3, every node that has value 1 at the start of round $f + 1$ broadcasts, causing every player who receives such a message to decide on 1. Since at most f players crash, we obtain the following as a consequence from this observation:

▶ **Lemma 8.** *If there is a round $r \leq f + 1$ such that $|P_r| \geq f + 1$, then the algorithm reaches agreement on 1.*

In the next lemma, we show that, as long as $|P_r|$ is small, that 1-valued message are not "wasted", in the sense that they only reach players that have not yet received such a message earlier on.

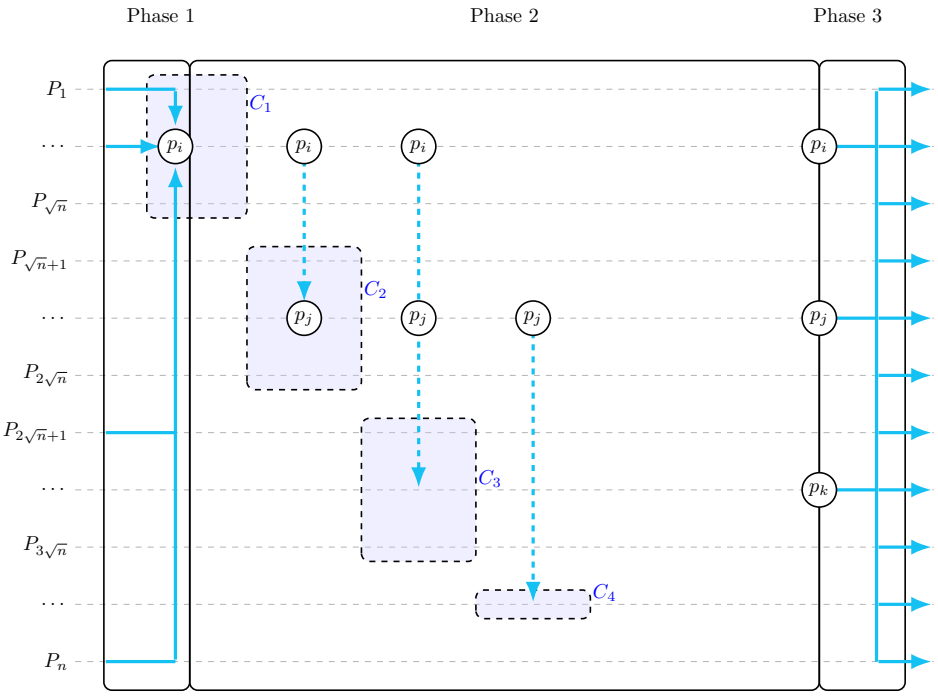


Figure 2 Binary Value Consensus execution

▶ **Lemma 9.** *If $|P_{r+1}| \leq f$ at the start of round $r+1 \leq f+1$, then there does not exist a player that has received a 1-valued message in two distinct rounds r' and r'' such that $r' < r'' \leq r$.*

Proof. Assume towards a contradiction that there is a set of players Q violating the conclusion of the lemma. That is, for each $p_i \in Q$, it holds that p_i updates $Y \leftarrow 1$ upon receiving a message in some round $r' < r$, and p_i also receives a message with value 1 in a later round $r'' \leq r$. Let $p_i \in Q$ be the player that minimizes the round number r'' over all players in Q, i.e., r'' is the first round in the execution in which some player receives a 1-valued message for the second time. Since p_i was awake in round r'' and received a 1-valued message, it follows that $p_i \in C_{r''}$. Recall that we created f committees in total, whereby the members of each committee C_h wake up in round h. Moreover, JoinCommittees (i.e. Algorithm 1) ensures that the indices of the committees that player p_i is a member of are spaced at least $\lceil\sqrt{n}\rceil$ apart, and thus $r'' \geq r' + \lceil\sqrt{n}\rceil$. According to the algorithm, p_i sends a message to every member of the currently awake committee in each round $r'+1, \ldots, r' + \left\lceil \frac{f+1}{\sqrt{n}} \right\rceil$, which yields a set M of $\lceil\sqrt{n}\rceil \cdot \left\lceil \frac{f+1}{\sqrt{n}} \right\rceil \geq f+1$ messages in total. Since r'' was the earliest round in which the lemma fails, it follows that every message in M sent by p_i in some round $s < r''$ is received by a node p_j that has not yet received a 1-valued message before round s. Note that only one of these messages could have been sent to $p_1 \in P_1$, which means that $p_j \notin P_s$ and $p_j \in P_{s+1}$, for every player $p_j \neq p_1$, who is the destination of a message in M. This tells us that $P_{r''}$ must contain at least $|M| - 1$ additional players, and thus

$$|P_{r+1}| \geq |P_{r''}| \geq |P_1| + (|M| - 1) \geq f + 1,$$

which yields the sought contradiction. ◀

▶ **Lemma 10.** *If a player sends a 1-valued message to all members of at least* $\left\lceil \frac{f+1}{\sqrt{n}} \right\rceil$ *committees by the end of round* f*, then* $|P_{f+1}| \geq f+1$*.*

Proof. Suppose that a player p does not crash before executing Lines 11-13 $\left\lceil \frac{f+1}{\sqrt{n}} \right\rceil$ times in some round t. We can assume that $|P_t| \leq f$, as otherwise the lemma follows immediately since $P_t \subseteq P_{f+1}$. Considering that each committee has a size of $\lceil \sqrt{n} \rceil$, it follows that they contain at least $\left\lceil \sqrt{f+1}/n \right\rceil \cdot \lceil \sqrt{n} \rceil \geq f+1$ players in total. By Lemma 9, all of these players are distinct and receive a 1-valued message for the first time. Moreover, p can be a member of at most one of these committees. Thus, we have $|P_{f+1}| \geq |P_{t+1}| \geq |P_1| + (f+1) - 1 \geq f+1$, as required. ◀

▶ **Lemma 11.** *It holds that either* $|P_{f+1}| \geq f+1$ *or the algorithm reaches agreement on* 0*.*

Proof. Recall that $P_1 = \{p_1\}$, where p_1 is a player who had input 1. By the code of the algorithm, p_1 attempts to send 1 to the members of the first $\left\lceil \frac{f+1}{\sqrt{n}} \right\rceil$ committees following C_1 in Phase 2, each of which has size $\geq \sqrt{n}$. Note that Phase 2 consists of rounds $2, \ldots, f$, and it holds that

$$f \geq \left\lceil \frac{f+1}{\sqrt{n}} \right\rceil - 1. \tag{1}$$

Thus, there are at least $\left(\left\lceil \frac{f+1}{\sqrt{n}} \right\rceil - 1 \right) \cdot \lceil \sqrt{n} \rceil$ players in the committees that are targeted by p_1 during Phase 2, which, together with the members of C_1, yields at least $\left\lceil \frac{f+1}{\sqrt{n}} \right\rceil \cdot \lceil \sqrt{n} \rceil$ players that will receive a message from p_1, unless it crashes. Similarly as in the proof of Lemma 9, we know that the JoinCommittees procedure ensures that p_1 is a member of at most one of these committees. It follows from Lemma 9 that no player is a member of more than one of these committees to which p_1 is sending a message to, and thus every p_j that receives a message from p_1 in some round r' has not received a 1-valued message in any earlier round, which ensures that $p_j \in P_{r'+1} \setminus P_{r'}$. Thus, $|P_{r'+1}| \geq |P_{r'}| + 1$ and, inductively,

$$|P_{r'+1}| \geq r' + 1. \tag{2}$$

By Lemma 10, we can assume that no player finishes forwarding all its 1-valued messages, which entails that p_1 must have crashed in some round $t \leq f$. Note that (2) implies $|P_{t+1}| \geq t+1$. Furthermore, for all $r' \in [t, f+1]$, we assume towards a contradiction that $|P_r| \leq f$. First, suppose there exists a round $r' \in [t, f+1]$ in which no 1-valued message is sent. It follows that every player who had $Y = 1$ or $X = 1$ must have crashed prior to round r'. This means that all remaining alive players have $Y = 0$ and $X = 0$, and hence they will reach agreement on 0.

Therefore, in the remainder of the proof, we can assume that some 1-valued message is sent in each round $r' \in [t, f+1]$. Note that each committee contains at least one player p' distinct from p_1 and since we assumed that $|P_{r'}| \leq f$, Lemma 9 ensures that $p' \in P_{r'+1} \setminus P_{r'}$. Applying this argument inductively for all rounds in $[t, f+1]$, completes the proof. ◀

The next lemma is a straightforward consequence of combining Lemmas 11 and 8:

▶ **Lemma 12.** *Algorithm 3 satisfies agreement.*

▶ **Lemma 13.** *Algorithm 3 sends* $\mathcal{O}(n^2)$ *messages of constant size and has an energy complexity of* $\mathcal{O}(\lceil f/\sqrt{n} \rceil)$*.*

Proof. For the message complexity, during the first phase, every player may send at most $\mathcal{O}(\sqrt{n})$ messages. During the second phase, every player may send at most $\mathcal{O}(\sqrt{n})$ messages for $\mathcal{O}(\frac{f}{\sqrt{n}})$ consecutive rounds, leading to $\mathcal{O}(f)$ messages for each player. In the last phase, each alive player sends $n-1$ messages. Overall, this yields a message complexity of $\mathcal{O}(n^2)$.

We next analyze the energy complexity. A given player p is awake in some round r due to one of the following @O reasons:

1. Player p belongs to committee C_r. As we have f committees of size $\Theta(\sqrt{n})$ to which the players are assigned to uniformly, each player belongs to at most $\mathcal{O}(\frac{f\cdot\sqrt{n}}{n})$ committees, which contributes $\mathcal{O}(\frac{f}{\sqrt{n}})$ to the number of awake rounds by that player.
2. Player p has input value 1 or receives a 1-valued message before round $f+1$. In that case, p stays awake for the next $\lceil\frac{f+1}{\sqrt{n}}\rceil$ consecutive rounds to forward the message to the subsequent committees. Note that this case can arise at most twice for p.

Overall, the energy complexity for each player is $\mathcal{O}\left(\frac{f}{\sqrt{n}}+\frac{f\cdot\sqrt{n}}{n}\right)=\mathcal{O}(\frac{f}{\sqrt{n}})$. ◀

We remark that our algorithm can work with committees of size k if players stay awake upon receiving a 1-valued message to inform the next $\frac{f}{k}$ committees. It is easy to see that $k=\sqrt{n}$ minimizes the energy complexity, which provides some intuition for our parameter choice for Algorithm 3. This procedure reaches a consensus with a better energy consumption at the cost of the binary input value restriction, while holding the bound of optimal round complexity. However, we observe a different message complexity in comparison with the multi-value consensus. This difference can be explained by the idea that during the last round, it is not clear who holds 1 and how many of those players need to be awake for ensuring agreement on 1. To address this issue, we need every player holding 1 to broadcast leading in the worst case to $\mathcal{O}(n^2)$ messages sent in the last rounds. In comparison, in multi-value consensus, the last committee of size $f+1$ contains sufficiently many players aware of the proposed decision value, leading to a message complexity of at most $\mathcal{O}(f.n)$ in this last round.

5 Conclusion

In this work, we studied consensus in the sleeping model under a crash-fault-tolerant setting, focusing on optimizing energy consumption while matching the known lower bound of $f+1$ rounds. By allowing players to alternate between active and inactive states, we demonstrated how energy-efficient consensus can be achieved while preserving correctness. We proposed protocols for both binary and multi-value consensus, achieving energy complexities of $E=\mathcal{O}(\frac{f}{\sqrt{n}})$ and $E=\mathcal{O}(\frac{f^2}{n})$, respectively.

References

Attiya and Welch(2004) Hagit Attiya and Jennifer Welch. 2004. *Distributed Computing: Fundamentals, Simulations and Advanced Topics (2nd ed.)*. John Wiley & Sons, Inc.

Barenboim and Maimon(2021) Leonid Barenboim and Tzalik Maimon. 2021. Deterministic Logarithmic Completeness in the Distributed Sleeping Model. In *DISC 2021, (LIPIcs, Vol. 209)*, Seth Gilbert (Ed.). LIPICS, 10:1–10:19. doi:10.4230/LIPICS.DISC.2021.10

Chang et al.(2018) Yi-Jun Chang, Varsha Dani, Thomas P. Hayes, Qizheng He, Wenzheng Li, and Seth Pettie. 2018. The Energy Complexity of Broadcast. In *PODC 2018*, Calvin Newport and Idit Keidar (Eds.). ACM, 95–104. doi:10.1145/3212734.3212774

Chang et al.(2020) Yi-Jun Chang, Varsha Dani, Thomas P. Hayes, and Seth Pettie. 2020. The Energy Complexity of BFS in Radio Networks. In *PODC 2020*, Yuval Emek and Christian Cachin (Eds.). ACM, 273–282. doi:10.1145/3382734.3405713

Chatterjee et al.(2020) Soumyottam Chatterjee, Robert Gmyr, and Gopal Pandurangan. 2020. Sleeping is efficient: MIS in O (1)-rounds node-averaged awake complexity. In *ACM PODC 2020*. 99–108.

Dolev and Strong(1983) Danny Dolev and H. Raymond Strong. 1983. Authenticated Algorithms for Byzantine Agreement. *SIAM J. Comput.* 12, 4 (1983), 656–666. doi:10.1137/0212045

Dufoulon et al.(2023) Fabien Dufoulon, William K. Moses Jr., and Gopal Pandurangan. 2023. Distributed MIS in O(log log n) Awake Complexity. In *PODC 2023, Orlando, FL, USA, June 19-23, 2023*. ACM, 135–145. doi:10.1145/3583668.3594574

Fischer et al.(1985) Michael J. Fischer, Nancy A. Lynch, and M. S. Paterson. 1985. Impossibility of Distributed Consensus with one Faulty Process. *J. ACM* 32, 2 (April 1985), 374–382.

Ghaffari and Portmann(2023) Mohsen Ghaffari and Julian Portmann. 2023. Distributed MIS with Low Energy and Time Complexities. In *PODC 2023, Orlando, FL, USA, June 19-23, 2023*. ACM, 146–156. doi:10.1145/3583668.3594587

Ghaffari and Trygub(2024) Mohsen Ghaffari and Anton Trygub. 2024. A Near-Optimal Low-Energy Deterministic Distributed SSSP with Ramifications on Congestion and APSP. In *PODC 2024, Nantes, France, June 17-21, 2024*, Ran Gelles, Dennis Olivetti, and Petr Kuznetsov (Eds.). ACM, 401–411. doi:10.1145/3662158.3662812

Hourani et al.(2022) Khalid Hourani, Gopal Pandurangan, and Peter Robinson. 2022. Awake-Efficient Distributed Algorithms for Maximal Independent Set. In *42nd IEEE International Conference on Distributed Computing Systems, ICDCS 2022, Bologna, Italy, July 10-13, 2022*. IEEE, 1338–1339. doi:10.1109/ICDCS54860.2022.00153

Lamport et al.(1982) Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems* 4, 3 (1982), 382–401.

Lynch(1996) Nancy Lynch. 1996. *Distributed Algorithms*. Morgan Kaufman Publishers, Inc., San Francisco, USA.

Pease et al.(1980) Marshall Pease, Robert Shostak, and Leslie Lamport. 1980. Reaching agreement in the presence of faults. *Journal of the ACM (JACM)* 27, 2 (1980), 228–234.

Xu et al.(2023) Jie Xu, Cong Wang, and Xiaohua Jia. 2023. A Survey of Blockchain Consensus Protocols. *ACM Comput. Surv.* 55, 13s (2023), 278:1–278:35. doi:10.1145/3579845